TRABAJO PRÁCTICO 1C

Nombres: Julieta Barahona Roth - Mia Bertola

Consignas:

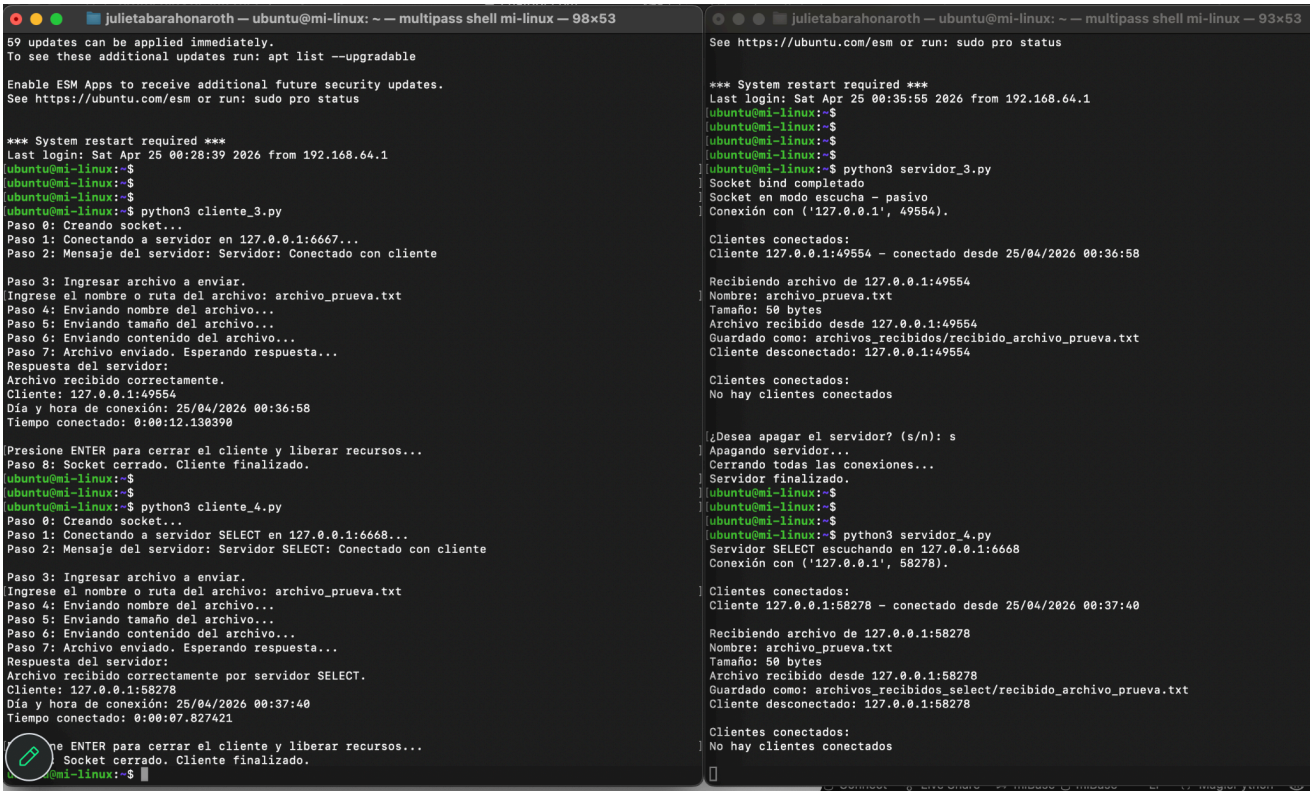
1. Por qué es necesario refrescar el servidor_con_interfaz.py para poder ver los mensajes?
2. ¿Funciona apagar el servidor ? ¿Cómo funciona ? ¿Qué hace ?
3. Escribir una aplicación cliente servidor en python que:
    - Envíe un archivo pequeño.
    - Que muestre las direcciones y puertos de todos los clientes conectados del lado del servidor.
    - Que devuelva a cada cliente el día y hora de conexión, y el tiempo que estuvo (o está) conectado.
4. Realizar la misma aplicación tanto para C-S con Concurrencia Aparente (Select) como C-S Concurrente.
5. De ser necesario agregar tiempo de espera, loop, sleep, con contadores para demorar los procesos.
6. Implementar una limpieza de recursos al salir del los programas (agregar opción de pregunta al usuario para cerrar los clientes).

Respuestas:

1. Es necesario refrescar el *servidor_con_interfaz.py* porque la interfaz gráfica no se actualiza automáticamente cuando llegan nuevos mensajes por el socket. Aunque el servidor recibe los datos correctamente mediante recv(), la UI funciona con un loop de eventos independiente que no detecta cambios externos a menos que se le indique explícitamente. Por eso, es necesario forzar una actualización (refresh) para mostrar los nuevos mensajes. Este problema se debe a la falta de sincronización entre la recepción de datos y la actualización de la interfaz, y suele resolverse utilizando threads, callbacks periódicos o mecanismos de comunicación entre procesos.

<meta http-equiv="refresh" content="3"> → con esta línea refrescamos la página cada 3 segundos.

2. Sí, apagar el servidor funciona correctamente. En este programa, al presionar el botón de apagado se activa un objeto *threading.Event* compartido (*apagado*), que actúa como una señal global entre los distintos hilos. Tanto el servidor TCP como los hilos que atienden a cada cliente verifican periódicamente si este evento está activado mediante *apagado.is_set()*. Cuando esto ocurre, el servidor deja de aceptar nuevas conexiones, sale de

su bucle principal y cierra el socket. Al mismo tiempo, cada hilo de cliente finaliza su ejecución y cierra su conexión. Esto permite un apagado ordenado del servidor TCP, liberando recursos y terminando las comunicaciones activas, mientras que la interfaz web de Flask continúa en ejecución.

Punto 3,4,5,6 → archivos .py adjuntos